



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 5: Exclusión mutua con Test-and-Set

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 17 de septiembre de 2026

Índice general

Índice de figuras	III
1. Exclusión mutua con Test-and-Set	1
1.1. Código fuente en Java	3
1.2. Código equivalente en C	5
1.3. Registro de ejecución	7
2. Conclusiones	9
Referencias	10

Índice de figuras

1.1. Registro de ejecución de la implementación en Java.	8
--	---

Listings

1.1. Implementación de Test-and-Set en Java	3
1.2. Implementación equivalente de Test-and-Set en C	5

1. Exclusión mutua con Test-and-Set

Desarrollen un programa en Java que cree un cerrojo utilizando AtomicBoolean, inicialmente en false; cree tres hilos que compartan el mismo cerrojo y un contador; implemente una operación para solicitar acceso a la sección crítica utilizando `getAndSet(true)`; mantenga al hilo en espera activa mientras el cerrojo esté ocupado; incremente el contador y muestre el identificador y el valor del contador dentro de la sección crítica; libere el cerrojo cambiándolo a false; y repita el proceso cinco veces por hilo.

Entradas. Tres hilos identificados como $P0$, $P1$ y $P2$, un AtomicBoolean lockFlag compartido e inicialmente en false, un contador compartido inicializado en cero y cinco iteraciones por hilo. La prueba de inicio permite que los tres hilos comiencen a solicitar el cerrojo de manera concurrente.

Procesamiento. La sección de entrada ejecuta requestCS. En Java, la expresión lockFlag.getAndSet(true) obtiene el valor anterior y cambia el cerrojo a true como una sola operación atómica. Si el valor anterior es true, el hilo permanece en el ciclo while y realiza espera activa. Si el valor anterior es false, el hilo entra a la sección crítica, incrementa el contador, muestra su identificador y libera el cerrojo con getAndSet(false). La implementación en C conserva la misma estructura mediante atomic_exchange sobre un atomic_bool.

Salidas. Cada entrada muestra el hilo, la iteración y el valor del contador. Como existen tres hilos y cada uno realiza cinco entradas, el valor esperado es $3 \times 5 = 15$. El valor obtenido en las pruebas fue 15 tanto en Java como en C.

Estructuras de control utilizadas. Se emplean tres hilos, un ciclo for para repetir las cinco entradas, un ciclo while para la espera activa, métodos de solicitud y liberación del cerrojo y una sección crítica entre las secciones de

entrada y salida. También se utilizan operaciones condicionales implícitas en el valor anterior que devuelve `getAndSet` o `atomic_exchange`.

Conceptos. Test-and-Set es una instrucción capaz de leer una variable y modificarla como una única operación atómica. Al convertir la comprobación y la modificación del cerrojo en una sola operación, solo un hilo puede obtener false cuando el cerrojo está libre. Los demás hilos obtienen true y permanecen en espera activa hasta que otro hilo libera el cerrojo. La espera activa consume tiempo de procesador, que es la limitación principal indicada en el módulo.

Secciones del programa.

- **Sección de entrada:** el método `requestCS` ejecuta repetidamente Test-and-Set.
- **Sección crítica:** el hilo incrementa el contador y muestra el resultado.
- **Sección de salida:** el método `releaseCS` cambia el cerrojo a false.
- **Sección restante:** la repetición de la prueba y la espera de los hilos al finalizar.

Prueba de escritorio. La siguiente secuencia representa el caso en que *P0* ya obtuvo el cerrojo y *P1* intenta entrar simultáneamente. Los valores se expresan como booleanos, de acuerdo con el enunciado.

Paso	Proceso	lockFlag antes	Operación	lockFlag después	Resultado
1	P0	false	TestAndSet(true) devuelve false	true	P0 entra a la sección crítica.
2	P1	true	TestAndSet(true) devuelve true	true	P1 no entra y comienza la espera activa.
3	P1	true	TestAndSet(true) devuelve true	true	P1 continúa esperando.
4	P0	true	TestAndSet(false)	false	P0 termina y libera el cerrojo.
5	P1	false	TestAndSet(true) devuelve false	true	P1 obtiene el cerrojo y entra.
6	P1	true	TestAndSet(false)	false	P1 termina y libera el cerrojo.

Evaluación. El algoritmo garantiza exclusión mutua porque la operación que obtiene el valor anterior y cambia el cerrojo es una operación atómica desde el punto de vista de los hilos que compiten. Cuando el cerrojo está libre, solo un hilo puede recibir false; los demás reciben true y no avanzan a la sección crítica. El contador final coincide con el esperado: 15.

Espera activa. La espera activa aparece en el ciclo `while` que contiene `getAndSet(true)` de Java y en el ciclo equivalente `while` que contiene `testAndSet(true)` de C. En cada repetición el hilo vuelve a comprobar el

cerrojo y cede brevemente el procesador, pero permanece listo para ejecutar. Esta solución evita la entrada simultánea, aunque consume CPU mientras existen hilos esperando.

1.1 Código fuente en Java

El siguiente listado conserva la forma del pseudocódigo del módulo: la solicitud usa `getAndSet(true)`, la liberación cambia el cerrojo a `false` y el contador se modifica dentro de la sección crítica.

```
1 import java.util.concurrent.CountDownLatch;
2 import java.util.concurrent.atomic.AtomicBoolean;
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Ejercicio01TestAndSet {
6     private static final int HILOS = 3;
7     private static final int ITERACIONES = 5;
8     private static final AtomicBoolean lockFlag = new AtomicBoolean
9         (false);
10    private static final AtomicInteger contador = new AtomicInteger
11        (0);
12    private static final CountDownLatch inicio = new CountDownLatch
13        (1);
14
15    // solicita acceso a la seccion critica mediante getAndSet
16    private static void requestCS(int id) {
17        while (lockFlag.getAndSet(true)) {
18            Thread.onSpinWait();
19        }
20    }
21
22    // libera el cerrojo al salir de la seccion critica
23    private static void releaseCS(int id) {
24        lockFlag.getAndSet(false);
25    }
26
27    // ejecuta las entradas del hilo a la seccion critica
```

Ejercicio 1. Exclusión mutua con Test-and-Set

```
25     private static void proceso(int id) {
26         try {
27             inicio.await();
28
29             for (int iteracion = 1; iteracion <= ITERACIONES;
30 iteracion++) {
31                 requestCS(id);
32
33                 int valor = contador.incrementAndGet();
34                 System.out.println("Hilo " + id
35 + " entra a la seccion critica en la
36 iteracion " + iteracion
37 + ", contador = " + valor);
38
39                 releaseCS(id);
40             }
41         } catch (InterruptedException excepcion) {
42             Thread.currentThread().interrupt();
43         }
44     }
45
46     public static void main(String[] args) throws
47 InterruptedException {
48         Thread[] hilos = new Thread[HILOS];
49
50         System.out.println("prueba de exclusion mutua con test-and-
51 set");
52
53         for (int id = 0; id < HILOS; id++) {
54             int idFinal = id;
55             hilos[id] = new Thread(() -> proceso(idFinal), "Hilo-"
56 + id);
57             hilos[id].start();
58         }
59
60         inicio.countDown();
61     }
62 }
```

```
56
57     for (Thread hilo : hilos) {
58         hilo.join();
59     }
60
61     System.out.println("contador esperado: " + (HILOS *
62     ITERACIONES));
63     System.out.println("contador obtenido: " + contador.get());
64     System.out.println("resultado: getAndSet garantiza
65     exclusion mutua con un cerrojo booleano");
66 }
```

Listing 1.1: Implementación de Test-and-Set en Java

1.2 Código equivalente en C

En C, `atomic_exchange` realiza la misma operación de lectura y modificación sobre `atomic_bool lockFlag`. La función `testAndSet` conserva el nombre conceptual del pseudocódigo y devuelve el valor anterior del cerrojo.

```
1  #include <pthread.h>
2  #include <sched.h>
3  #include <stdatomic.h>
4  #include <stdbool.h>
5  #include <stdio.h>
6
7  #define HILOS 3
8  #define ITERACIONES 5
9
10 static atomic_bool lockFlag = ATOMIC_VAR_INIT(false);
11 static atomic_int contador = ATOMIC_VAR_INIT(0);
12
13 // simula la operacion test-and-set del pseudocodigo
14 static bool testAndSet(bool nuevoValor) {
15     return atomic_exchange_explicit(&lockFlag, nuevoValor,
16     memory_order_acq_rel);
17 }
```

Ejercicio 1. Exclusión mutua con Test-and-Set

```
16 }
17
18 // solicita acceso a la seccion critica mediante espera activa
19 static void requestCS(int id) {
20     (void) id;
21
22     while (testAndSet(true)) {
23         sched_yield();
24     }
25 }
26
27 // libera el cerrojo al salir de la seccion critica
28 static void releaseCS(int id) {
29     (void) id;
30     testAndSet(false);
31 }
32
33 // ejecuta las entradas del hilo a la seccion critica
34 static void *proceso(void *argumento) {
35     int id = *(int *) argumento;
36
37     for (int iteracion = 1; iteracion <= ITERACIONES; ++iteracion)
38     {
39         requestCS(id);
40
41         int valor = atomic_fetch_add_explicit(&contador, 1,
42         memory_order_relaxed) + 1;
43         printf("Hilo %d entra a la seccion critica en la iteracion
44         %d, contador = %d\n",
45         id, iteracion, valor);
46
47         releaseCS(id);
48     }
49
50     return NULL;
51 }
```

```
49
50 int main(void) {
51     pthread_t hilos[HILOS];
52     int identificadores[HILOS];
53
54     printf("prueba de exclusion mutua con test-and-set\n");
55
56     for (int id = 0; id < HILOS; ++id) {
57         identificadores[id] = id;
58         pthread_create(&hilos[id], NULL, proceso, &identificadores[
59         id]);
60     }
61
62     for (int id = 0; id < HILOS; ++id) {
63         pthread_join(hilos[id], NULL);
64     }
65
66     printf("contador esperado: %d\n", HILOS * ITERACIONES);
67     printf("contador obtenido: %d\n", atomic_load_explicit(&
68     contador, memory_order_relaxed));
69     printf("resultado: test-and-set garantiza exclusion mutua con
70     un cerrojo booleano\n");
71 }
```

Listing 1.2: Implementación equivalente de Test-and-Set en C

1.3 Registro de ejecución

La salida depende del orden en que el planificador concede el procesador a cada hilo. Lo importante es que cada entrada ocurre con un valor distinto y que el contador termina en 15.

Ejercicio 1. Exclusión mutua con Test-and-Set

```
prueba de exclusion mutua con test-and-set
Hilo 2 entra a la seccion critica en la iteracion 1, contador =
    1
Hilo 1 entra a la seccion critica en la iteracion 1, contador =
    2
Hilo 0 entra a la seccion critica en la iteracion 1, contador =
    3
Hilo 1 entra a la seccion critica en la iteracion 2, contador =
    4
Hilo 0 entra a la seccion critica en la iteracion 2, contador =
    5
Hilo 2 entra a la seccion critica en la iteracion 2, contador =
    6
...
contador esperado: 15
contador obtenido: 15
resultado: getAndSet garantiza exclusion mutua con un cerrojo
booleano
```

Figura 1.1: Registro de ejecución de la implementación en Java.

2. Conclusiones

La implementación demuestra la función de Test-and-Set dentro de la exclusión mutua. El cerrojo booleano inicia libre, cada hilo solicita acceso con una única operación que obtiene el valor anterior y escribe true, y el hilo que logra entrar libera el cerrojo al salir. Por ello, tres hilos que realizan cinco entradas producen exactamente quince incrementos del contador.

La prueba de escritorio muestra la diferencia con una solución que primero comprueba y después modifica el cerrojo. Con Test-and-Set esas dos acciones se realizan como una operación atómica, así que dos hilos no pueden obtener false para la misma entrada. Su costo es la espera activa, que consume CPU mientras el cerrojo permanece ocupado.

Bibliografía

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.